

Scenario 04 — Clone Systems SEUXDR

Assessor guide & answer key

This document is **not** for the engineer running the pilot. It contains the verified findings, the commands that prove each one, **what a successful deliverable looks like**, and the review rubric.

It differs from the instructor guides in scenarios 01–03 in one important way: there are no *seeded* vulnerabilities here. Every finding is a real property of a real commercial product, so the answer key is a **verification key** — a set of commands that let you confirm each finding independently rather than take the pilot's word for it.

1. Pilot topology

- **VM1** — CyberFort (`access.cyber-fort.eu` , tenant `CRA Extended` , CRA mode enabled).
- **VM2** — SEUXDR manager stack: front end `:8080` , manager API + agent WebSocket `:8443` , agent enrolment `:8081` (mTLS), Ollama `:11434` . 32 GB RAM, GPU recommended.
- **VM3** — endpoint host(s) running the SEUXDR agent.

The hosted CyberFort instance cannot reach a private lab subnet, so the pilot's core evidence comes from the three **source-side** tools — Code Analysis, Dependency Check, SBOM Generator — which accept a ZIP or a GitHub URL. Network and application scanning of VM2 is optional and requires a range-local CyberFort. **Do not mark the pilot down for absent Nmap/ZAP output** unless a range-local instance was available.

2. Findings answer key

Findings are numbered as they appear in `PILOT_RUNBOOK.md` . All line references are `t4.4_siem_ai_remediation @ 00a95ad` .

#	Finding	Where	Detected by	CyberFort CRA objective(s)
1	Annex III Class I classification removes the self-assessment route	product function, three independent limbs	CRA Scope Assessment wizard	Chapter III · Chapter I
2	No SBOM ships with the product	absent; <code>licenses/credits.txt:5</code> still holds the template placeholder	SBOM Generator (Syft)	VH-1
3	Direct dependencies on unreleased and pre-module versions, unscanned	<code>go.mod:5–35</code> · <code>manager_front/package.json:13,17,18</code>	OSV Dependency Check	VH-1 · VH-2 · Annex I 2
4	Live Wazuh/OpenSearch admin credential committed in three tracked files	<code>main.go:176–177</code> + two integration tests	Semgrep <code>hardcoded–password</code>	Annex I 3a · 3c · 2
5	Group KEK private key stored unwrapped beside the keys it wraps	<code>manager/api/configuration_service/configuration_service.go:130,134,142</code> → <code>groups.key_encryption_key</code>	Semgrep + schema review	Annex I 3c

#	Finding	Where	Detected by	CyberFort CRA objective(s)
6	Unauthenticated remote destructive action across the agent fleet	route <code>manager/routes/routes.go:86</code> ; handler <code>manager/handlers/agent_actions.go:385-388,454</code>	Semgrep + route review + the product's own Postman collection	Annex I 3b · 2 · 3d
7	Unvalidated LLM output executed as root; prompt injection unmitigated	<code>react_service.go:99-101,318-320</code> → <code>agent/monitoring/monitoring.go:446-484</code> ; prompt <code>prompt_service.go:206</code>	manual data-flow review	Annex I 1 · 3d · 3i
8	Entire Annex I Part II apparatus absent	no <code>SECURITY.md</code> , no CI, no tags; <code>readme.md:394</code> empty placeholder	repository review against the Technical File checklists	VH-1 ... VH-8
9	DoC readiness baseline 7%, "Not Yet Ready" (tenant-aggregated; 2/30 objectives compliant for SEUXDR alone); no single product version to declare	agent <code>1.0.1</code> , front end <code>0.0.0</code> , manager unversioned	EU Declaration of Conformity page	Chapter II Art. 13

A tenth item is worth raising in review even though the runbook does not number it: **the product has three names** — `SEUXDR` (Go module, `go.mod:1`), `CyberGuard` (documentation and logo) and `SECUR-EU SIEM / NEXT GEN SIEM` (shipped UI, `manager_front/index.html:7`, `headerBar.tsx:28`). A Declaration of Conformity needs one.

3. Verification commands — with expected output

Run these from the root of a clone of `t4.4_siem_ai_remediation`. They are read-only.

3.1 No authentication middleware exists (Finding #6, backlog BLOCK-1)

```

wc -l manager/middlewares/middleware.go
grep -rn "func.*Auth\|Bearer\|jwt.Parse" manager/middlewares/
grep -n "m.CustomLogger\|Use(" manager/routes/routes.go | head

```

Expected output (key lines): `middleware.go` is **13 lines** and the grep returns nothing. The `/api` group applies only `m.CustomLogger`. The absence of any output from the second command is the finding — there is no authentication middleware to find.

3.2 No user, role or session schema (backlog BLOCK-1)

```

grep -rniE "create table (users|roles|sessions|permissions)" manager/database/migrations/
grep -rn "api/login" manager/routes/routes.go manager/handlers/
grep -rn "DROP TABLE users" manager/database/migrations/

```

Expected output: nothing for the first two. The third returns `000001_init_mg.down.sql:7` — a vestigial drop of a table that is never created, left behind when authentication was removed. That single line is the cleanest proof that the removal was deliberate rather than accidental.

3.3 The remediation endpoint is unauthenticated and unattributed (Finding #6)

```
grep -n "execute-action" manager/routes/routes.go
sed -n '380,395p;450,458p' manager/handlers/agent_actions.go
grep -n "ACTION_MODE" manager/handlers/agent_actions.go
python3 -c "import json;d=json.load(open('SEUXDR_Manager_API.postman_collection.json'));print('auth block present:', 'auth' in d)"
```

Expected output: the route is registered on the group that carries only the logger. The handler validates `BLOCK_IP` / `DELETE_FILE` / `KILL_PROCESS` and dispatches. `ACTION_MODE` returns no hits in this file — the `manual_only` safety mode is never consulted on this path. The Postman collection declares no auth on any of its 31 requests, which is consistent with the code and is the honest way to demonstrate the finding live.

3.4 The dead safety path (backlog HIGH-6)

```
grep -rn "ExecuteActiveResponseCommand" agent/ manager/ --include=*.go
grep -rn "generateOSSpecificCommand\|activeResponseService.SendCommand" manager/ --include=*.go
grep -n "exec.Command\|CommandContext" agent/monitoring/monitoring.go
```

Expected output: `ExecuteActiveResponseCommand` — which holds the whitelist, the blocklist and the timeout — is called only from the `command` -message branch. `generateOSSpecificCommand` and `SendCommand` appear only at their definitions: **the manager never emits a command message**, so the documented protections never execute. `monitoring.go` uses `exec.Command`, never `CommandContext` — no timeout on the live path.

3.5 Target validation is non-emptiness only (Finding #7)

```
sed -n '95,105p' manager/api/socanalystservice/react_service.go
sed -n '456,470p' agent/monitoring/monitoring.go
grep -rniE "protected.?(path|process)|denylist|blocklist.*path|canonicaliz" manager/ agent/ --include=*.go
```

Expected output: both validation sites check only that the trimmed target is non-empty. The third command returns **nothing** — there is no protected-path or protected-process list anywhere in the codebase.

3.6 Prompt injection surface (Finding #7)

```
grep -n "FullLog\|{{\." manager/api/socanalystservice/prompt_service.go | head -20
```

Expected output: `{{.FullLog}}` interpolated via `text/template`, which escapes for HTML/text contexts, not for prompt contexts. Attacker-controlled log content reaches the model verbatim.

3.7 Secrets and permissions (Findings #4, #5)

```
grep -rn "S1lMfWxh1HB6SbKg" . --include=*.go
grep -n "os.ModePerm" manager/main.go manager/crons/log_cleanup.go agent/comms/tlsconfig.go
grep -n "os.Create" manager/helpers/helpers.go
grep -n "0644\|0666" manager/api/websocket/service/logstore.go manager/api/configuration/service/configuration_service.go
grep -n "api_key\|license_key" manager/handlers/agent_actions.go | head -3
```

Expected output: the credential appears in **three** tracked files. `os.ModePerm` (`0777`) is used for `MkdirAll` on the storage and certificate directories. `os.Create` defaults to `0666` before umask, so private keys land `0644`. Log and `keys.json` writes are explicitly `0644`. Line 62 of `agent_actions.go` logs both enrolment secrets in cleartext.

3.8 No SBOM, no CI, no tags (Findings #2, #8)

```
find . -iname "*sbom*" -o -iname "*.cdx.json" -o -iname "*spdx*" -o -iname "bom.xml" | grep -v node_modules
ls -a .github .gitlab-ci.yml Jenkinsfile .circleci 2>&1 | tail -2
ls SECURITY.md CHANGELOG.md 2>&1 | tail -1
git tag | wc -l
grep -n "contact \[\\]" readme.md
grep -n "SecureEU/seuxdr" CyberGuard_Stripped_Documentation.md
```

Expected output: no SBOM artefact. No CI configuration of any kind. No `SECURITY.md`, no `CHANGELOG.md`. `git tag | wc -l` returns `0`. `readme.md:394` contains the literal `contact []`. The documentation directs vulnerability reports to a repository that is not this product's.

3.9 Inert safety knobs (backlog MED-2)

```
grep -rn "confidence" manager/ agent/ --include=*.go | grep -v _test | wc -l
grep -rn "cooldown\|CooldownPeriod" manager/ --include=*.go | grep -v config | grep -v _test
grep -n "size\": 10000\|\"size\":" manager/api/opensearchservice/opensearch_service.go
grep -n "MinRuleLevel" manager/api/messageprocessor/message_processor.go
```

Expected output: `confidence` returns `0` non-test hits despite `manager.yaml:55-67` configuring confidence thresholds. `cooldown` appears only in config plumbing — there is no timestamp comparison that would enforce it. The OpenSearch query hard-codes `"size": 10000`, ignoring `max_alerts_batch`. `MinRuleLevel` is used only to build a prompt string.

3.10 The genuine strengths — verify these too (Claims 1, 2, 4, 6)

An assessor who only verifies the failures has not assessed the product.

```
grep -rn "aes.NewCipher\|cipher.NewGCM" agent/ manager/ --include=*.go | wc -l
grep -rniE "NewCBCEncrypter|ECB|PKCS1v15" agent/ manager/ --include=*.go | wc -l
grep -rn "MinVersion" manager/main.go manager/mtls/mtls.go agent/comms/tlsconfig.go
grep -rn "InsecureSkipVerify" agent/ --include=*.go | wc -l
grep -rn "RequireAndVerifyClientCert" manager/main.go
grep -rn "4096" manager/mtls/mtls.go
ls manager/database/migrations/000005_failed_alert_queue.up.sql
```

Expected output: AES-GCM appears throughout; **CBC, ECB and PKCS#1 v1.5 return `0`**. `MinVersion` is set on every server and client. `InsecureSkipVerify` returns `0` for the entire agent tree — the four occurrences are all manager→loopback. Client certificates are required on the enrolment listener. The runtime PKI uses RSA-4096. The failed-alert queue migration exists and is substantial. These confirm Claims 1, 2, 3, 4 and 9 and are the basis for the two Yes answers.

4. What a successful deliverable looks like in CyberFort

Use these as grading anchors. A pilot that produces scanner output but not these artefacts has not completed.

4.1 Asset record

`SEUXDR` registered as `Software Product`, `Economic Operator: Manufacturer`, **Criticality set to the verbatim option "Security information and event management (SIEM) systems"** under the *ANNEX III – Class I* group header, with a written justification citing Article 32(2). A pilot that leaves Criticality blank has skipped the single most consequential field, because the CRA Technical File pages, the CE Marking Checklist and the DoC readiness score all key off it.

4.2 Risk register

At least **five** new risks scoped to `SEUXDR`, covering: the unauthenticated destructive endpoint, absent authentication, unvalidated LLM output, the missing Part II apparatus, and the secrets exposure. Each should carry a Likelihood/Severity pair, a treatment status, and a linked CRA objective. Findings should be connected via **Link to Risk** from Scan Findings, not filed as orphans.

4.3 Conformity assessment

All **52** questions answered with `Completed: YES`, each with an evidence description and, where scanner output exists, an attachment. The expected distribution is **2 Yes / 19 Partially / 31 No**.

The two Yes answers are the discriminator. A pilot that answers everything `No` has produced a bug list, not a conformity assessment, and has thrown away the product's real strengths. Q29 (availability after an incident) and Q34 (security-relevant information recording) are genuinely defensible and must be claimed with evidence. Conversely, a pilot claiming more than a handful of `Yes` answers has inflated the baseline — check Q13, Q20 and Q21 first, since those three cannot honestly be anything but `No` in this build.

4.4 Objectives checklist

All **30** objectives in the **SEUXDR** asset scope given a compliance status, with evidence uploaded. Expected: **2 compliant, 10 partially, 18 not compliant**. Annex I **3f** and **3j** are the two compliant ones.

4.5 Artefacts that did not exist before the pilot

- A machine-readable **SBOM** — the product shipped none.
- A **CycloneDX VEX** export.
- Populated **CRA Technical File** pages across all six areas.
- A **gap analysis** PDF with the three gap categories enumerated.
- A **DoC readiness score** on the record.

4.6 The remediation backlog

Ranked, with the notified-body decision (BLOCK-0) and the disclosure policy (BLOCK-4) identified as the cheapest high-leverage moves. A backlog that leads with the low-severity code hygiene items has mis-prioritised.

5. Review rubric (100 points)

Activity	Points
CRA scope determined; Annex III Class I justified in writing, including the Article 32(2) consequence	10
Asset registered with complete CRA metadata (Criticality, Vertical Profile, Economic Operator, justification)	8
CRA framework seeded; 30 objectives present in the asset scope	5
SBOM generated and attached to the asset record	8
Dependency Check and Code Analysis run against the real source; findings interpreted, not just pasted	10
Five or more blocking risks filed and linked from Scan Findings	10
VEX statements published and exported	5
All six CRA Technical File areas reviewed; evidence uploaded to the Evidence Library	8
All 52 conformity questions answered with evidence	12
Both defensible Yes answers claimed with evidence (Q29, Q34)	5
All 30 objectives given a compliance status	8
Gap analysis exported; compliance chain traced end to end for at least one risk	5
DoC readiness score recorded; CE Marking Checklist created	3
Remediation backlog ranked with the blockers correctly identified	8
Use of AI Suggest Answers / Suggest from Scans / AI Auto-select at least once	5

Pass threshold: 70.

Deduct up to 10 for an inflated baseline — answering **Partially** where the evidence supports only **No** is the most common and most damaging error, because it produces a remediation plan that fixes the wrong things.

6. Things that are *not* findings

Do not accept these as non-conformities if the pilot reports them:

- **action_mode: "manual_only"** as the shipped default is a strength, not a gap. It is the correct secure-by-default posture for an autonomous-response product (`manager/manager.yaml:49` , enforced at `message_processor.go:1176,1191`). The gap is that three code paths bypass it and that an `unset` key defaults to `automatic` — not the default itself.
- **InsecureSkipVerify on the manager's loopback calls to OpenSearch and the Wazuh API** is a weakness worth noting but not a finding of substance: all four occurrences target `127.0.0.1` inside the same container. The material point is that it is

unconditional and non-configurable.

- **Per-agent remediation scoping is not a limitation.** The absence of any fleet-wide action primitive is a deliberate and structural blast-radius control (`message_processor.go:430`), and it is the strongest safety property in the design.
- **The attack-simulation scripts are not vulnerabilities.** All eleven inject synthetic log entries via the host's native logging system. No packets leave the host and no real malware is created.
- **The bundled Wazuh dependency is not itself a finding.** Bundling a SIEM engine is a legitimate architectural choice; the finding is that the installer is fetched and executed **without checksum or signature verification** (`Dockerfile:53` , `startup.sh:19`).

7. Known operational quirks

Symptom	Cause	Fix / expectation
Web UI shows only a login page that never succeeds	No <code>/api/login</code> route exists in the stripped build; <code>PrivateRoute</code> gates every page on a token that can never be issued	Drive the manager through the Postman collection. This is the honest demo path and it <i>is</i> Finding #6
Nothing executes after an attack simulation	<code>action_mode: "manual_only"</code> — analyses land as <code>manual_action_required</code>	Expected. This is Claim 7, not a fault
AI analysis times out	Configured <code>analysis_timeout: 360</code> is overridden by a hard-coded 60 s context at <code>message_processor.go:1065</code>	Use a GPU host
~60 s before anything happens	30 s alert polling, and the poller reads 30 s into the past to let Wazuh catch up	Expected
Flood of low-severity analyses	<code>min_rule_level</code> is configured but never enforced	Backlog MED-2
Only the most recently blocked IP appears in <code>pfctl</code> on macOS	<code>seuxdr-firewall-macos.sh:44</code> replaces the whole <code>pf</code> anchor on each block	Backlog HIGH-5. Demonstrate <code>BLOCK_IP</code> on Linux or Windows
CRA Technical File pages 404 on direct URL entry	They resolve only through in-app navigation	Navigate via Documents → Technical File in the sidebar

8. Reset between pilot runs

On the product host:

```
cd /srv/seuxdr
sh clear-db.sh          # removes manager/storage/manager.db and agent/storage/agent.db
sh delete-certs.sh      # wipes manager/certs, agent/certs, manager_front/certs, agent_info.enc
docker compose down && docker compose up --build -d
```

In CyberFort, delete the pilot's assessment and risk entries, or scope a fresh assessment to a new asset version. **Do not delete the asset record** — `soc_analyst_analyses` and `soc_analyst_actions` in the product database carry `ON DELETE CASCADE` on `agent_id` (backlog MED-3), and the equivalent caution applies to the platform's compliance chain: deleting the asset orphans the objective statuses and evidence you just produced.